

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1.Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. (5 marks)

Problem Statement

Design a Reinforcement Learning framework where an agent learns the best actions by interacting with an environment to maximize cumulative rewards.

Objective

- Understand the agent–environment interaction.
- Explain states, actions, rewards, and cumulative rewards.

Eligibility

Basic knowledge of Python and Reinforcement Learning concepts.

Challenge Rules

- Define the RL components.
- Explain the interaction cycle.
- Describe cumulative reward.

Programming Language

Python

```
import random

states = ["Start", "Middle", "Goal"]
actions = ["Left", "Right"]
state = "Start"
total_reward = 0

print("Simple RL Framework\n")

for step in range(5):

    action = random.choice(actions)

    if state == "Start" and action == "Right":

        state = "Middle"

        reward = 5

    elif state == "Middle" and action == "Right":

        state = "Goal"

        reward = 10

    else:

        reward = -1

    total_reward += reward

    print(f"Step {step+1}")

    print("Action :", action)

    print("State :", state)

    print("Reward :", reward)

    print()
```

```
if state == "Goal":  
    break  
  
print("Total Reward =", total_reward)
```

Environment Setup

- Python 3.12
- Gymnasium
- NumPy

Input

Current state of the environment.

Output

 `print("Total Reward =", total_reward)`

... Simple RL Framework

```
Step 1  
Action : Left  
State  : Start  
Reward : -1
```

```
Step 2  
Action : Left  
State  : Start  
Reward : -1
```

```
Step 3  
Action : Right  
State  : Middle  
Reward : 5
```

```
Step 4  
Action : Left  
State  : Middle  
Reward : -1
```

```
Step 5  
Action : Left  
State  : Middle  
Reward : -1
```

```
Total Reward = 1
```

Constraints

- Sequential decision making.
- Rewards may be delayed.

- Agent learns through trial and error.

RL Framework

Environment



Reward, State



Agent



Action

Agent–Environment Interaction

1. Agent observes the current state.
2. Agent selects an action.
3. Environment executes the action.
4. Agent receives reward.
5. Agent updates its policy.
6. Process repeats until the goal is reached.

RL Components

Component	Description
State	Current situation
Action	Decision taken
Reward	Feedback from environment
Policy	Strategy used by the agent

Importance of Cumulative Reward

Instead of maximizing immediate reward, RL maximizes the **sum of future rewards**, resulting in better long-term decisions.

Evaluation Criteria

- Correct RL framework
- Proper explanation of states, actions, rewards
- Understanding of cumulative reward

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. (5 marks)

Problem Statement

Model a decision-making problem as a Markov Decision Process (MDP).

Objective

Understand the components of an MDP.

Eligibility

Basic RL knowledge.

Challenge Rules

Explain each MDP component.

Programming Language

Python

Coding Challenge 2

Simple Markov Decision Process

```
states = ["A", "B", "Goal"]
```

```
transition = {
```

```
    "A": "B",
```

```
    "B": "Goal"
```

```
}
```

```
reward = {
```

```
    "A": 5,
```

```
"B": 10,

"Goal": 20

}

state = "A"

print("Markov Decision Process\n")

while state != "Goal":

    print("Current State :", state)

    next_state = transition[state]

    print("Next State   :", next_state)

    print("Reward       :", reward[next_state])

    print("-----")

    state = next_state

print("Goal Reached")
```

Environment Setup

Python + NumPy

Input

Current state.

Output

```
print("Reward       :", reward[next_state])

print("-----")

state = next_state

print("Goal Reached")
```

```
... Markov Decision Process
```

```
Current State : A
Next State    : B
Reward        : 10
```

```
-----
Current State : B
Next State     : Goal
Reward         : 20
```

```
-----
Goal Reached
```

Constraints

Decision depends only on the current state.

MDP Components

Component	Description
State (S)	Current condition
Action (A)	Possible decisions
Transition Probability (P)	Probability of moving to another state
Reward (R)	Immediate feedback
Discount Factor (γ)	Importance of future rewards

MDP Flow

State



Action



Next State



Reward

Example

State = Robot at Position A

Action = Move Right

Transition Probability = 0.9

Reward = +10

Discount Factor = 0.9

Evaluation Criteria

- Correct MDP components
- Proper example
- Understanding of transition probability

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. (5 marks)

Problem Statement

Explain the role of policy and value functions using the Bellman Equation.

Objective

Understand policy evaluation and long-term reward estimation.

Eligibility

Basic RL concepts.

Challenge Rules

Explain policy, value functions, and Bellman equation.

Programming Language

Python

```
# Coding Challenge 3
```

```
# Bellman Equation
```

```
gamma = 0.9
```

```
reward = 10
```

```
future_value = 20
```

```
current_value = reward + gamma * future_value
```

```
print("Bellman Equation")
```

```
print("Reward =", reward)
```

```
print("Future Value =", future_value)
```



```
print("Discount Factor =", gamma)
print("Current State Value =", current_value)
```

Environment Setup

Python

Input

Current state.

Output

```
print("Current State Value =", current_value)
```

```
Bellman Equation
Reward = 10
Future Value = 20
Discount Factor = 0.9
Current State Value = 28.0
```

Constraints

Future rewards are discounted.

Policy

A policy determines which action should be selected in every state.

Value Functions

State Value

$V(s)$

Expected long-term reward from a state.

Action Value

$Q(s,a)$

Expected reward after taking an action.

Bellman Equation

$V(s) = R + \gamma V(s')$

where

- R = Reward

- γ = Discount Factor
- $V(s')$ = Future State Value

Importance

Bellman Equation repeatedly updates state values until the optimal policy is obtained.

Evaluation Criteria

- Correct Bellman Equation
- Proper explanation of policy
- Correct value function definitions

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency. (5 marks)

Problem Statement

Compare exploration and exploitation strategies.

Objective

Understand the exploration–exploitation trade-off.

Eligibility

Basic RL knowledge.

Challenge Rules

Compare ϵ -greedy and softmax.

Programming Language

Python

Coding Challenge 4

Epsilon Greedy

import random

epsilon = 0.2

Q = [5, 8, 2, 10]

```
print("Epsilon Greedy\n")
for i in range(10):
    if random.random() < epsilon:
        action = random.randint(0, 3)
        strategy = "Exploration"
    else:
        action = Q.index(max(Q))
        strategy = "Exploitation"
    print("Step", i+1)
    print("Action :", action)
    print("Strategy :", strategy)
    print()
```

Environment Setup

Python

Input

Current Q-values.

Output



```
... Epsilon Greedy

Step 1
Action : 3
Strategy : Exploitation

Step 2
Action : 3
Strategy : Exploitation

Step 3
Action : 3
Strategy : Exploitation

Step 4
Action : 3
Strategy : Exploitation

Step 5
Action : 3
Strategy : Exploitation

Step 6
Action : 3
Strategy : Exploitation

Step 7
Action : 3
Strategy : Exploitation

Step 8
Action : 3
Strategy : Exploitation

Step 9
Action : 3
Strategy : Exploitation

Step 10
Action : 2
Strategy : Exploration
```

Constraints

Balance exploration and exploitation.

Exploration

Try new actions to gather information.

Exploitation

Select the action with the highest estimated reward.

Comparison

ϵ -Greedy	Softmax
Random exploration	Probability-based exploration
Simple implementation	Better action selection
Fast	Smooth exploration

Advantages

ϵ -Greedy

- Easy implementation
- Low computation

Softmax

- Better exploration
- Less random behaviour

Evaluation Criteria

- Correct comparison
- Trade-off explanation
- Advantages and limitations

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. (5 marks)

Problem Statement

Illustrate how Q-learning updates the action-value function.

Objective

Understand Q-learning updates and convergence.

Eligibility

Basic RL knowledge.

Challenge Rules

Explain Q-learning update equation.

Programming Language

Python

Coding Challenge 5

Q-Learning Update

alpha = 0.5

gamma = 0.9

Q = 10

reward = 5

future_Q = 20

new_Q = Q + alpha * (reward + gamma * future_Q - Q)

print("Q-Learning")

print("Old Q Value :", Q)

print("Reward :", reward)

print("Future Q :", future_Q)

print("Updated Q :", round(new_Q,2))

Environment Setup

Python

Input

Current state and action.

Output

```
print("Updated Q :", round(new_Q,2))  
  
*** Q-Learning  
Old Q Value : 10  
Reward : 5  
Future Q : 20  
Updated Q : 16.5
```

Constraints

Learning rate α and discount factor γ .

Q-Learning Update Equation

$$Q(s,a)=Q(s,a)+\alpha[R+\gamma\max_{a'}Q(s',a')-Q(s,a)]$$

where

- α = Learning Rate
- γ = Discount Factor
- R = Reward

Example

Current Q-value = 10

Reward = 5

Maximum Future Q = 20

Learning Rate = 0.5

Discount Factor = 0.9

Updated Q-value

$$Q=10+0.5[(5+0.9\times 20)-10]$$

$$Q=10+0.5(23-10)$$

$$Q=16.5$$

Convergence

Repeated updates gradually improve the Q-values until they converge to the optimal action-value function.

Evaluation Criteria

- Correct Q-learning equation
- Proper numerical example
- Explanation of convergence
- Action-Value Function

Code of Conduct:

I S.Sirivally / 192472371 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sirivally.S

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately